

<Educa+>

Documento de Arquitetura de Software

Versão <2.1>

Histórico da Revisão

Data	Versão	Descrição	Autor
<25/04/2026>	<1.0>	<Criação Inicial do Documento>	<Erick Henrique>
<26/04/2026>	<2.0>	<Tópicos adicionais do Documento >	<Alberto Gomes Paiva>
<04/05/2026>	<2.1>	<Revisão e correção do Documento>	<Felipe Oliveira>

Índice Analítico

1.	Introdução	3
1.1	Finalidade	3
1.2	Escopo	3
1.3	Definições, Acrônimos e Abreviações	3
2.	Restrições e requisitos arquitetuais	3
3.	Visão de Casos de Uso	4
4.	Visão Lógica	4
4.1	Representação do domínio da aplicação	4
4.2	Decisões arquitetuais	4
4.3	Representação da arquitetura lógica	5
4.4	Representação do funcionamento da arquitetura	6
5.	Visão de Processos (opcional)	6
6.	Visão da Implementação (opcional)	6
7.	Visão de Implantação	6
8.	Visão de Dados	6
9.	Volume e Desempenho	6
10.	Referências	7

Documento de Arquitetura de Software

1. Introdução

O presente documento descreve a arquitetura do sistema Educa+, apresentando uma visão geral de sua estrutura, organização e funcionamento. O Educa+ é uma aplicação de apoio pedagógico voltada ao ensino médio, com foco no acompanhamento da aprendizagem dos alunos, organização de conteúdos alinhados à BNCC e suporte à tomada de decisão do professor.

A arquitetura do sistema foi definida com base nos requisitos funcionais e não funcionais levantados, bem como nas necessidades identificadas no Documento de Visão. O sistema busca apoiar o professor na identificação de dificuldades de aprendizagem, geração de recomendações personalizadas e melhoria contínua do desempenho dos alunos, sem substituir sua atuação pedagógica.

Este documento segue uma abordagem baseada no modelo de visões arquiteturais, permitindo a compreensão do sistema sob diferentes perspectivas.

1.1 Finalidade

Este documento oferece uma visão geral arquitetural abrangente do sistema, usando diversas visões arquiteturais para representar diferentes aspectos do sistema. O objetivo deste documento é capturar e comunicar as decisões arquiteturais significativas que foram tomadas em relação ao sistema. O documento irá adotar uma estrutura baseada na visão “4+1” de modelo de arquitetura, conforme (Kruchten, 1994).

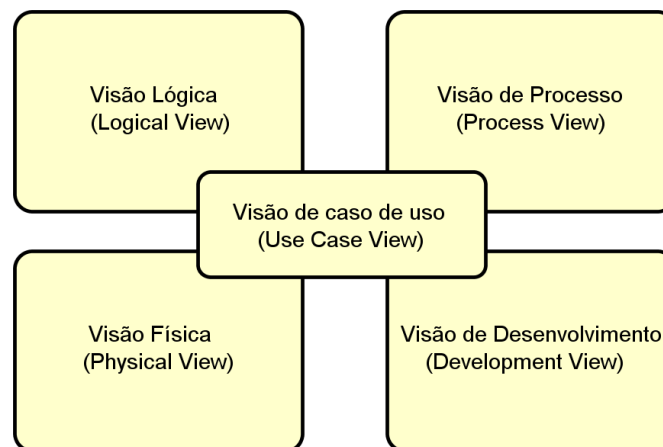


Figura 1 - Visões 4+1 da arquitetura de software (Kruchten, 1994)

Este documento tem como finalidade apresentar uma visão arquitetural completa do sistema Educa+, descrevendo suas principais decisões de projeto, organização estrutural e funcionamento.

Ele serve como referência para a equipe de desenvolvimento e demais stakeholders, auxiliando na compreensão da solução, na manutenção do sistema e na tomada de decisões técnicas ao longo do projeto.

1.2 Escopo

O sistema Educa+ é uma aplicação educacional que tem como objetivo apoiar professores do ensino médio no acompanhamento da aprendizagem dos alunos.

O sistema contempla funcionalidades como cadastro do contexto pedagógico, criação e aplicação de atividades avaliativas, consolidação de desempenho, identificação de dificuldades, envio de feedback, geração de recomendações personalizadas e sugestão de conteúdos complementares, incluindo o uso controlado de inteligência artificial.

O sistema atende principalmente dois perfis de usuários: professores e alunos, garantindo controle de acesso e privacidade dos dados.

1.3 Definições, Acrônimos e Abreviações

- BNCC: Base Nacional Comum Curricular
- HU: História de Usuário
- RN: Regra de Negócio
- RNF: Requisito Não Funcional
- IA: Inteligência Artificial
- LGPD: Lei Geral de Proteção de Dados
- Educa+: Sistema de apoio pedagógico

2. Restrições e requisitos arquiteturais

Atributo de qualidade	Requisito arquitetural	Solução
Desempenho	O sistema precisa responder em até 3 segundos nas consultas de desempenho e recomendações para 90% das requisições em condições normais de uso (RNF-002).	Optamos pelo NestJS com Node.js 20, que lida bem com múltiplas requisições ao mesmo tempo sem travar. As consultas mais pesadas, como consolidação de desempenho por aluno, são pré-calculadas e salvas na tabela DESEMPENHO_ALUNO, então na hora de exibir os dados não precisa recalcular tudo do zero. O Prisma cuida do acesso ao banco com índices nas colunas mais consultadas.

Interoperabilidade	O sistema precisa se comunicar com o Firebase e ser consumido pelo app mobile em Android. (RNF-005).	A API foi construída no padrão REST com respostas em JSON, que é o formato mais compatível com o React Native. A comunicação com o Firebase Authentication e o Firebase Cloud Messaging é feita pelos próprios SDKs oficiais do Google, o que simplifica bastante a integração.	
Usabilidade	A interface tem que ser simples e direta, permitindo que um professor consiga fazer o cadastro inicial em até 5 minutos, com pelo menos 85% de sucesso em testes com usuários reais (RNF-001).	O app foi desenvolvido em React Native com Expo, priorizando telas limpas e fluxos curtos. A ideia foi pensar no professor do ensino médio que não necessariamente tem familiaridade com tecnologia, então evitamos sobrecarregar as telas com informações desnecessárias.	
Confiabilidade	Toda ação importante no sistema precisa ser registrada com identificação do usuário, data, hora e o que foi feito. As sugestões geradas por IA também precisam ser rastreáveis para que o professor possa revisá-las depois (RNF-006, RNF-007).	Usamos a biblioteca Winston no backend para registrar os eventos que realmente importam: login e logout, criação e edição de questões, aplicação de atividades, feedbacks e geração de sugestões. Erros do sistema também ficam registrados. Não logamos tudo para não gerar um volume absurdo de dados sem utilidade. Os logs ficam armazenados por 12 meses.	
Segurança	O sistema precisa garantir que cada usuário acesse apenas o que é dele, com senhas protegidas e dados acadêmicos seguros, respeitando a LGPD (RNF-003, RNF-004).	As senhas nunca são salvas como texto puro — passam pelo bcrypt antes de ir pro banco. No login, o sistema gera um token JWT com validade de 24 horas, que o app envia em toda requisição.	

		O NestJS verifica esse token e o perfil do usuário antes de liberar qualquer operação. Todo o tráfego acontece via HTTPS.	
Facilidade de manutenção	O código precisa ser organizado de forma que qualquer integrante do grupo consiga entender e dar manutenção sem depender de uma pessoa só.	A estrutura do NestJS divide o código em controllers, services e repositories, o que deixa cada parte com uma responsabilidade clara. O TypeScript ajuda bastante nisso também, porque pega boa parte dos erros antes mesmo de rodar o código. As migrations do Prisma garantem que qualquer alteração no banco fique registrada e versionada.	
Portabilidade	O app tem que funcionar bem em Android, sem precisar manter dois projetos separados (RNF-005).	O React Native com Expo resolve isso — um único código-base que gera o app para a Android. Durante o desenvolvimento, o Expo Go permite testar direto no celular sem precisar de emulador nem de configuração nativa complicada.	
Escalabilidade	O sistema precisa aguentar crescer ao longo dos semestres, começando com o piloto de até 3 escolas e podendo expandir depois.	O backend foi feito sem guardar estado em memória, então escalar horizontalmente é simples se necessário. O banco MySQL foi modelado de forma normalizada com índices pensados para o crescimento estimado de 80 MB por semestre.	
Disponibilidade	O sistema precisa estar no ar durante o horário de aula, de segunda a sexta das 07h às 22h, com pelo menos 95% de disponibilidade nesse período.	O backend será hospedado em nuvem com monitoramento de uptime. O banco tem backup diário para garantir que nenhum dado seja perdido em caso de falha. O Firebase, por ser gerenciado pelo Google, já tem alta disponibilidade por	

		natureza.	
--	--	-----------	--

3. Visão de Casos de Uso

O sistema Educa+ é estruturado com base em histórias de usuário que representam suas funcionalidades principais.

As principais histórias de usuário são:

- HU1 Cadastrar contexto pedagógicoProfessor
- HU2 Consolidar desempenho da turma e dos alunosProfessor
- HU3 Enviar feedback do professor ao alunoProfessor
- HU4 Gerar recomendações personalizadasSistema
- HU5 Permitir resposta do aluno ao feedbackAluno
- HU6 Criar e aplicar atividades avaliativasProfessor
- HU7 Consultar mapa de desempenhoProfessor
- HU8 Gerar questões de reforço com IAProfessor
- HU9 Visualizar desempenho individualAluno
- HU10 Sugerir conteúdos complementaresAluno
- HU11 Responder atividade avaliativaAluno
- HU12 Gerenciar banco de questõesProfessor
- HU13 Cadastrar escolaProfessor
- HU14 Gerenciar alunos da turmaProfessor
- HU15 Editar e excluir atividades não aplicadasProfessor
- HU16 Consultar histórico de atividades por turmaProfessor
- HU17 Visualizar histórico de atividades respondidasAluno
- HU18 Realizar atividade de reforçoAluno
- HU19 Realizar login no sistemaProfessor e Aluno
- HU20 Recuperar senhaProfessor e Aluno
- HU21 Utilizar assistente de IA para sugestão de questõesProfessor

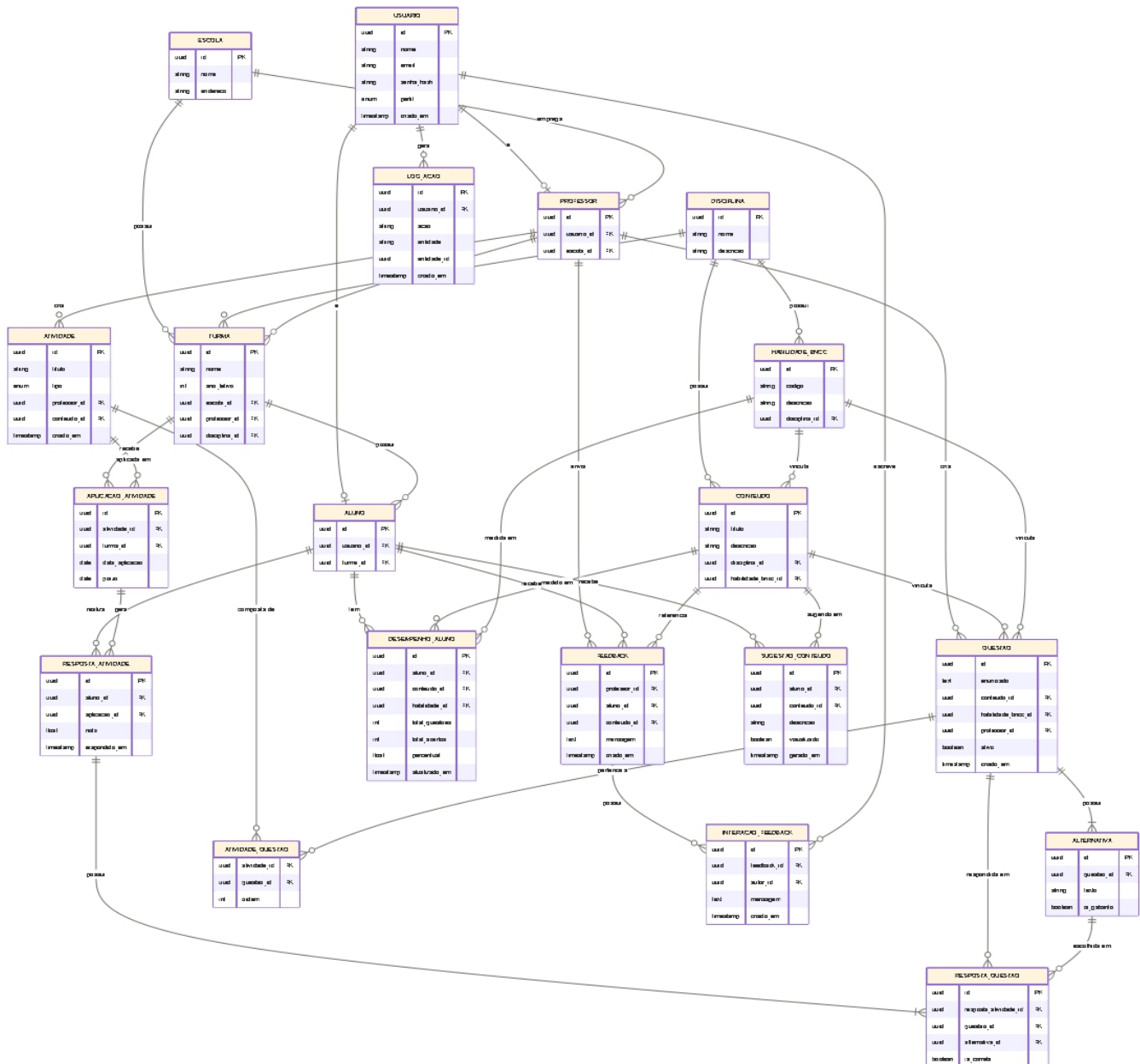
Essas funcionalidades refletem a jornada do usuário professor e aluno, contemplando desde o cadastro inicial até o acompanhamento contínuo da aprendizagem.

4. Visão Lógica

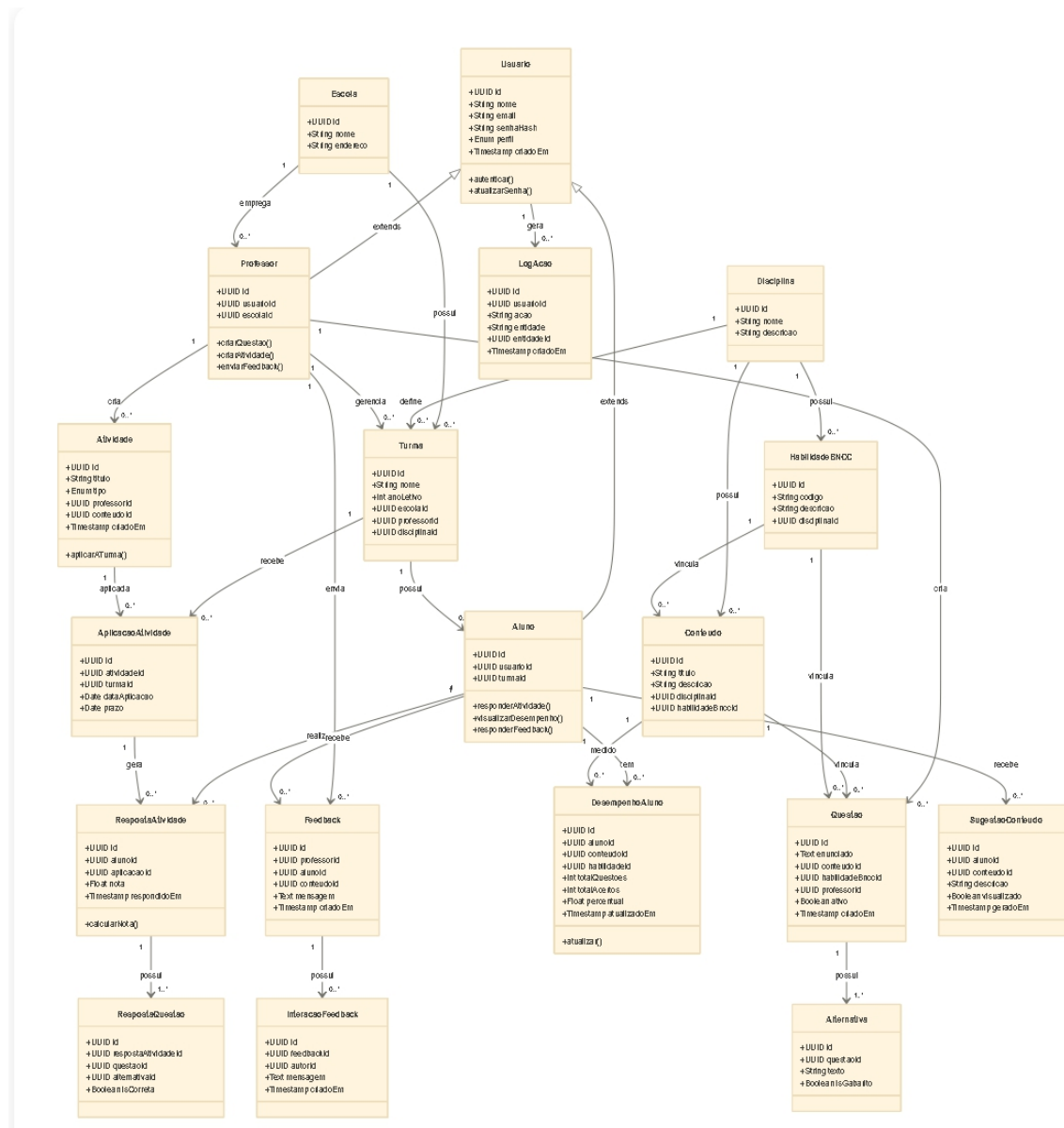
A arquitetura lógica do sistema Educa+ é organizada de forma modular, separando responsabilidades em diferentes camadas para facilitar manutenção, escalabilidade e clareza estrutural.

4.1 Representação do domínio da aplicação

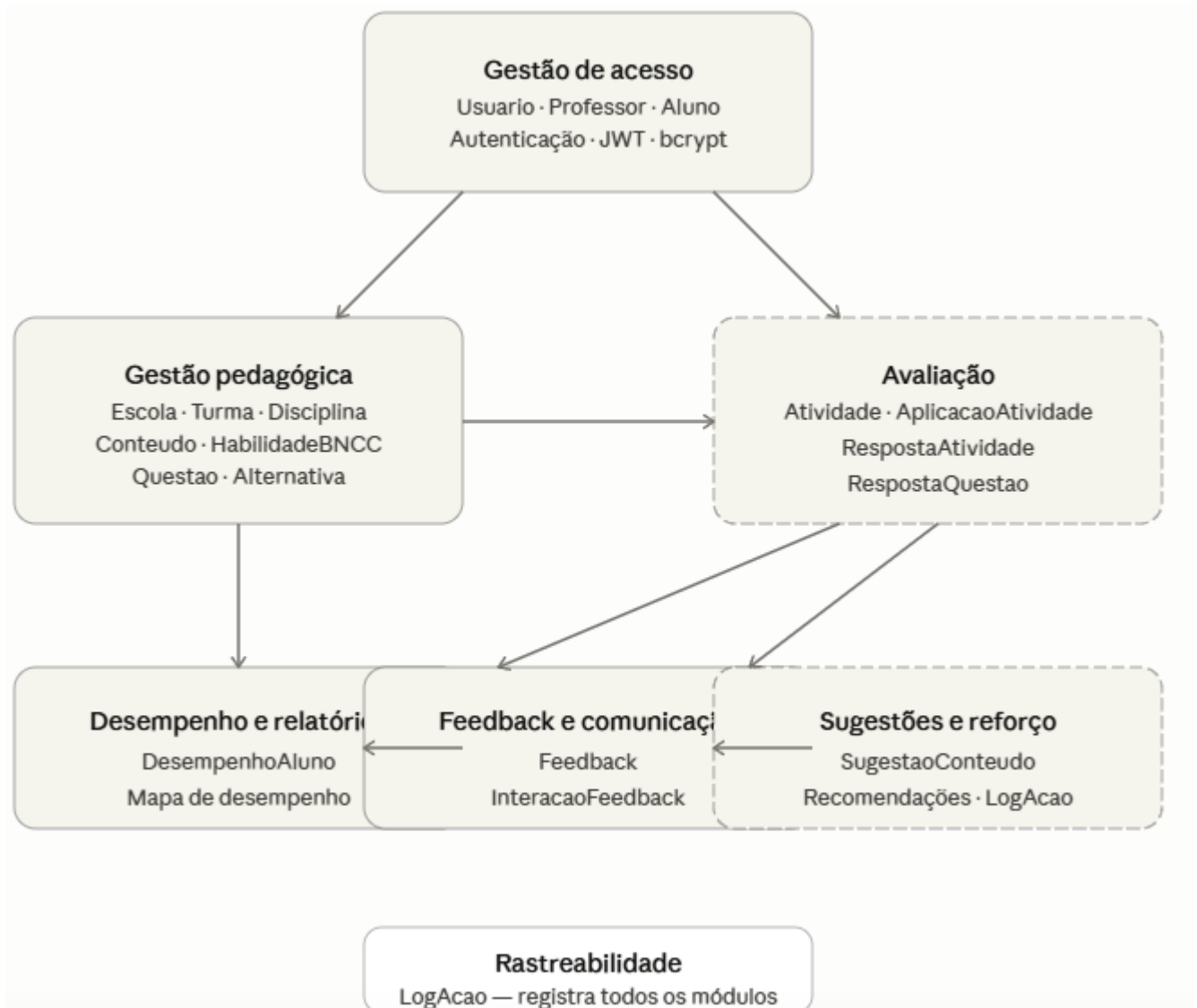
O diagrama de classes da UML é uma das principais ferramentas para representar a estrutura de um sistema de software. Ele mostra as classes que compõem o sistema, seus atributos, métodos e como elas se relacionam entre si. No contexto do Educa+, o diagrama de classes foi fundamental para organizar e deixar claro como as diferentes partes do sistema se conectam — desde o cadastro de usuários e turmas até o registro de respostas e a geração de sugestões para os alunos. Com ele, ficou mais fácil para a equipe entender o domínio da aplicação antes de partir para o desenvolvimento.



A figura a seguir apresenta o diagrama de pacotes com a modularização da visão de negócio do sistema Educa+. Diferente do diagrama de classes, que mostra a estrutura interna de cada entidade, o diagrama de pacotes agrupa as funcionalidades em módulos com responsabilidades bem definidas, facilitando a compreensão do sistema como um todo. O Educa+ foi organizado em seis módulos principais: Gestão de Acesso, Gestão Pedagógica, Avaliação, Desempenho e Relatórios, Feedback e Comunicação, e Sugestões e Reforço. As setas entre os módulos indicam as dependências, mostrando como o resultado de um módulo alimenta o funcionamento do próximo.



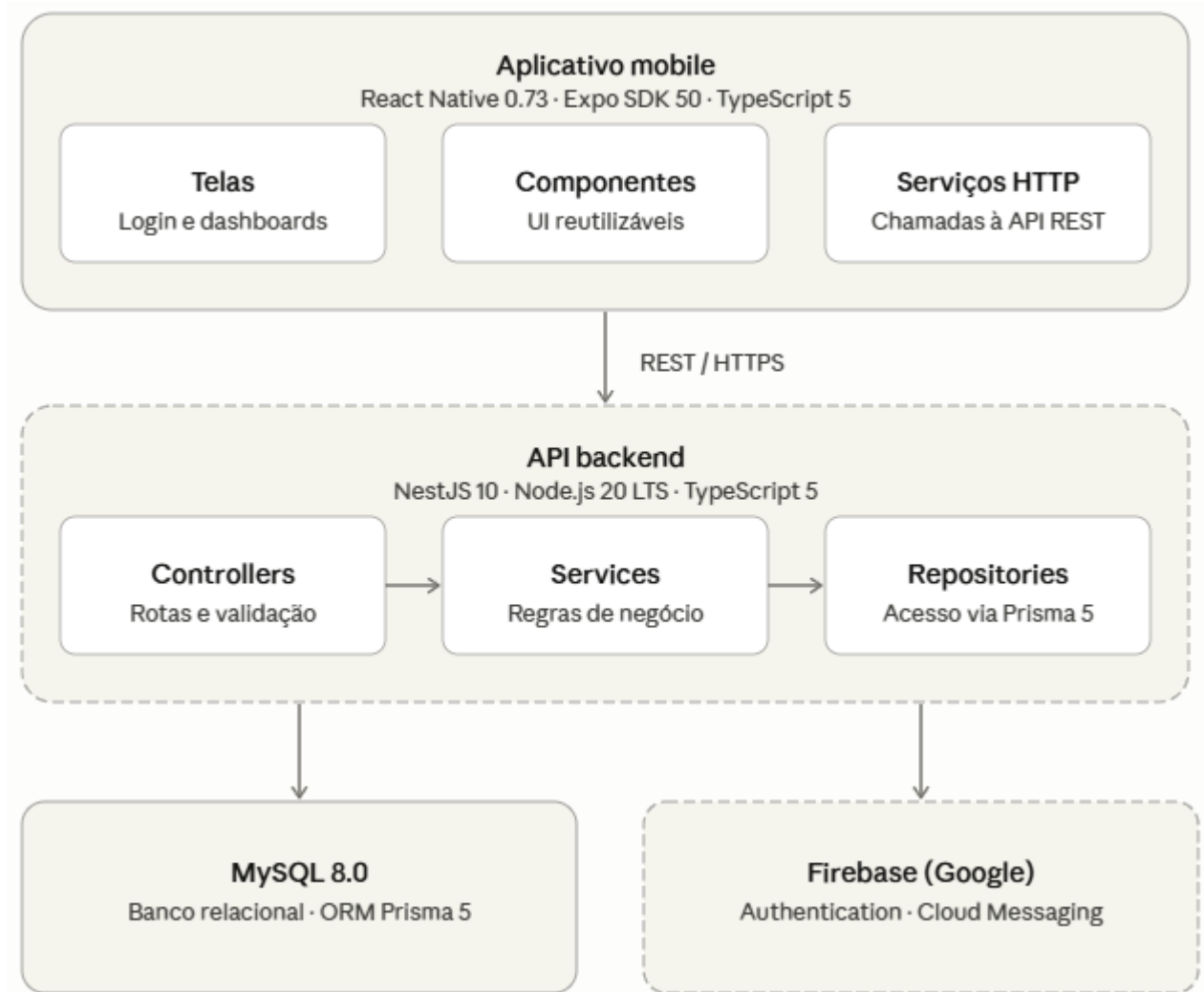
Além do diagrama de classes, a figura a seguir apresenta o diagrama de pacotes com a modularização da visão de negócio do sistema, organizando as funcionalidades em grupos com responsabilidades bem definidas.



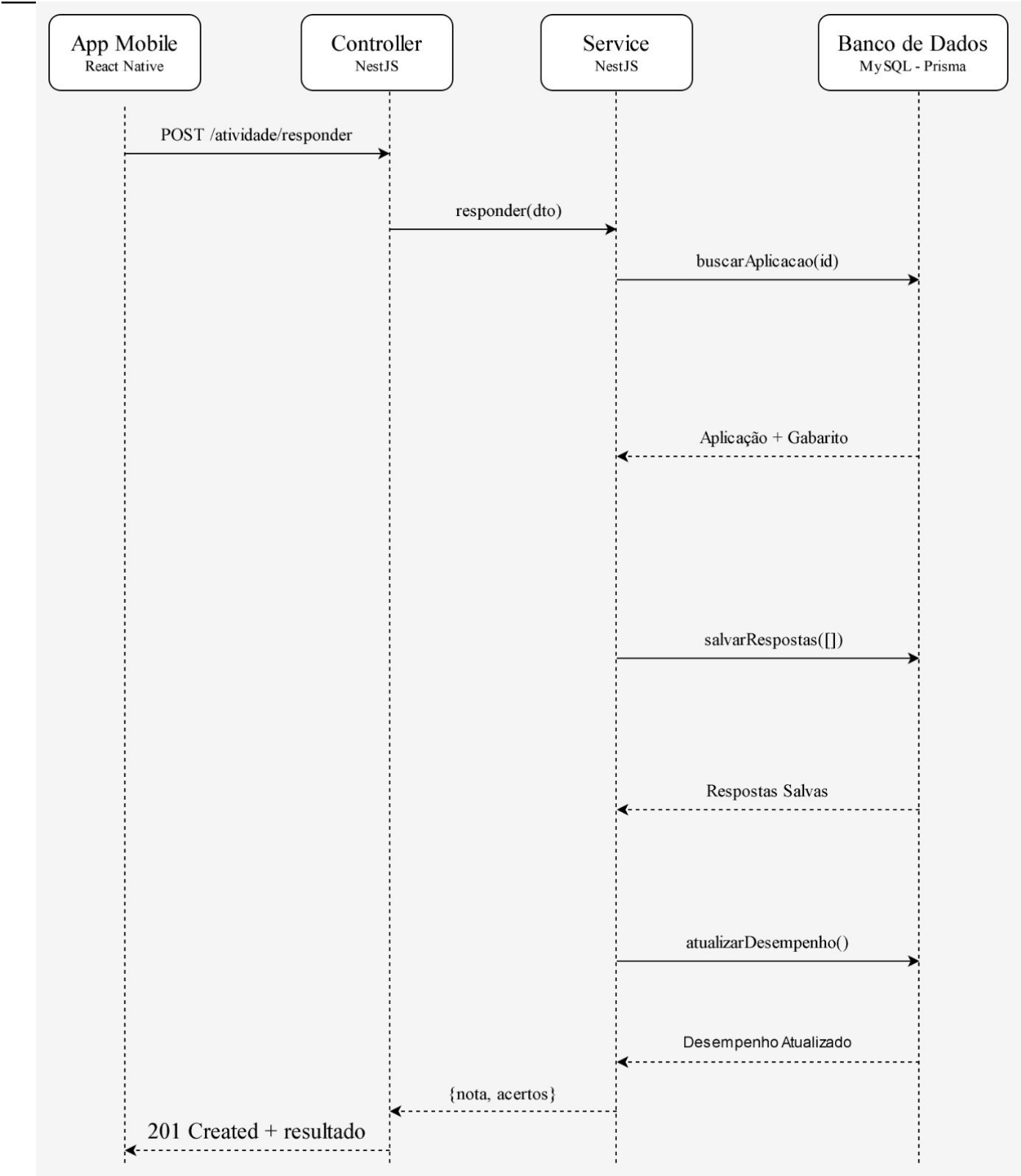
4.2 Decisões arquiteturais

Item arquitetural	Decisão	Justificativa
Estilo arquitetural	Arquitetura em camadas com API REST	O sistema é dividido em três camadas — apresentação (app mobile), lógica de negócio (backend NestJS) e persistência (MySQL). Essa separação garante independência entre as partes, facilita manutenção e permite que o app mobile e o backend evoluam de forma independente, sem acoplamento direto entre interface e banco de dados.
Linguagem de programação	TypeScript 5.x (mobile e backend)	O TypeScript adiciona tipagem estática ao JavaScript, reduzindo erros em tempo de execução e tornando o código mais previsível. Isso é especialmente importante para um sistema com regras de negócio complexas como consolidação automática de desempenho, controle de acesso por perfil e rastreabilidade de ações.
Framework mobile	React Native 0.73 com Expo SDK 50	O React Native permite o desenvolvimento de um único código-base para Android, atendendo ao RNF-005. O Expo simplifica o processo de build e teste em Android sem necessidade de configuração nativa, e o Expo Go permite testes em dispositivos reais durante o desenvolvimento sem necessidade de emulador.
Framework backend	NestJS 10 com Node.js 20 LTS	O NestJS oferece uma estrutura de camadas bem definida — controllers, services e repositories — que organiza de forma natural as regras de negócio do Educa+. Seus decorators nativos simplificam a implementação de autenticação JWT, controle de acesso por perfil e validação de dados. A integração nativa com TypeScript garante consistência com o restante do projeto.
Formas de persistência	ORM Prisma 5	O Prisma gera queries tipadas a partir do schema do banco, evitando SQL manual e reduzindo riscos de inconsistência entre o modelo de dados e o código da aplicação. Suas migrations controlam a evolução do schema de forma rastreável ao longo do desenvolvimento.
Banco de dados	MySQL 8.0	banco relacional amplamente utilizado, com suporte robusto a transações, boa integração com o ORM Prisma e familiaridade consolidada da equipe de desenvolvimento, o que reduz a curva de aprendizado e agiliza a implementação. Atende à exigência de banco relacional definida para o projeto.
Autenticação	JWT com bcrypt (fator de custo 12)	A autenticação é realizada por usuário e senha. A senha nunca é armazenada em texto puro — ela é processada pelo algoritmo bcrypt com fator de custo 12 antes de ser salva no banco, atendendo ao RNF-003 e RNF-004. A cada login bem-sucedido, o sistema gera um token JWT assinado com chave secreta e validade de 24 horas. O token é enviado pelo app em toda requisição no cabeçalho Authorization, e o backend valida a assinatura e o perfil do usuário antes de processar qualquer operação, garantindo o controle de acesso por perfil definido na RN-009.
Serviços de terceiros	Firebase (Google) Firebase Auth + FCM	O Firebase Authentication centraliza o gerenciamento de sessões de usuário, reduzindo a complexidade de implementação de autenticação segura. O Firebase Cloud Messaging (FCM) é utilizado para envio de notificações push ao app mobile, alertando alunos sobre novos feedbacks disponíveis e atividades aplicadas pelo professor.
Logs	Winston 3.x	Biblioteca de logging para Node.js utilizada no backend para registro dos seguintes eventos: autenticação (login, logout e tentativas falhas), criação e edição de questões e atividades, aplicação de atividades a turmas, envio e resposta de feedbacks, e geração de sugestões de conteúdo. Erros de sistema são capturados com stack trace completo. Operações de leitura simples não são registradas para evitar volume excessivo, mantendo o foco na rastreabilidade exigida pelo RNF-006 e RNF-007.

4.3 Representação da arquitetura lógica



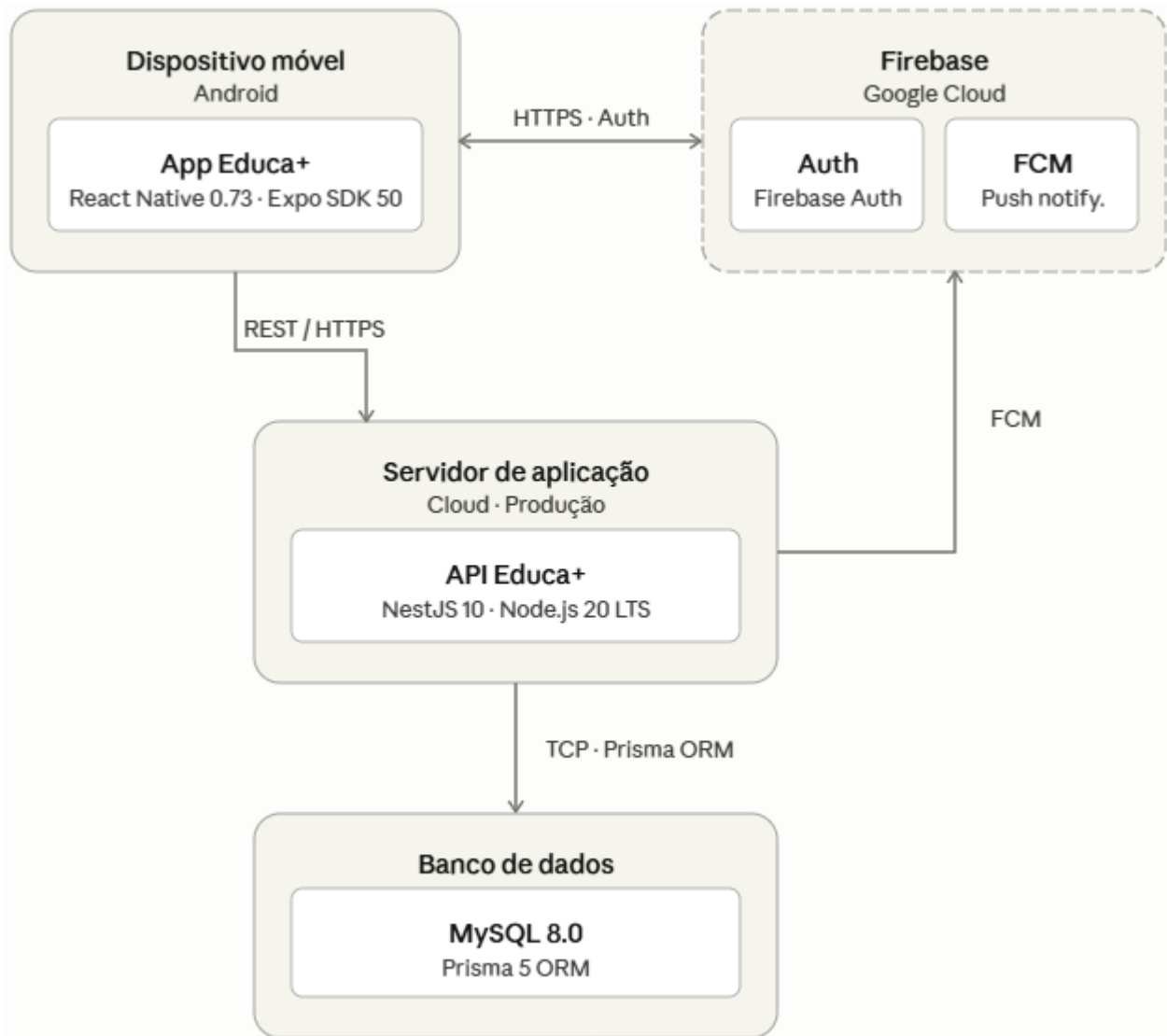
4.4 Representação do funcionamento da arquitetura



5. Visão de Processos (opcional)

6. Visão da Implementação (opcional)

7. Visão de Implantação



8. Visão de Dados

A persistência do Educa+ é integralmente relacional, gerenciada pelo MySQL 8.0 via ORM Prisma 5. O sistema não utiliza armazenamento de arquivos locais — todos os dados são estruturados em tabelas relacionais com integridade referencial garantida por chaves estrangeiras. O modelo de dados completo está representado no Modelo Entidade-Relacionamento (MER) elaborado na seção 4.1 deste documento.

Minimundo

O sistema é utilizado por dois perfis de usuário: professores e alunos. Todo usuário é registrado na entidade **USUARIO**, que armazena credenciais de acesso com senha protegida por hash bcrypt. A partir

do perfil, o usuário é especializado em **PROFESSOR** ou **ALUNO**.

Cada professor está vinculado a uma **ESCOLA** e pode gerenciar uma ou mais **TURMA**. Cada turma pertence a uma escola, é regida por um professor e está associada a uma **DISCIPLINA**. Os alunos são matriculados em turmas.

As disciplinas possuem **HABILIDADE_BNCC** cadastradas conforme os códigos e competências da Base Nacional Comum Curricular. Cada **CONTEUDO** está obrigatoriamente vinculado a uma disciplina e a uma habilidade da BNCC, garantindo a rastreabilidade pedagógica exigida pelo sistema.

O professor cria **QUESTAO** de múltipla escolha, cada uma vinculada a um conteúdo e a uma habilidade da BNCC. Cada questão possui exatamente quatro registros em **ALTERNATIVA**, sendo um deles marcado como gabarito. As questões compõem uma biblioteca reutilizável.

O professor cria **ATIVIDADE** — avaliativa ou de reforço — e seleciona questões da biblioteca para compô-la, registradas com ordem definida em **ATIVIDADE_QUESTAO**. A atividade é então aplicada a uma turma por meio de **APLICACAO_ATIVIDADE**, que registra a data de aplicação e o prazo de resposta.

Quando o aluno responde uma atividade, é criado um registro em **RESPOSTA_ATIVIDADE** com a nota calculada automaticamente. Cada resposta individual a uma questão é armazenada em **RESPOSTA_QUESTAO**, com referência à alternativa escolhida e ao indicador de acerto.

O sistema consolida automaticamente o desempenho em **DESEMPENHO_ALUNO**, agrupado por aluno, conteúdo e habilidade, com percentual de acertos atualizado a cada nova correção.

O professor pode enviar mensagens de feedback ao aluno por meio de **FEEDBACK**, vinculadas a um conteúdo específico. O histórico bidirecional de trocas entre professor e aluno é mantido em **INTERACAO_FEEDBACK**.

Com base no desempenho identificado, o sistema gera registros em **SUGESTAO_CONTEUDO** associando o aluno aos conteúdos que necessitam de reforço.

Por fim, toda ação relevante no sistema — autenticações, cadastros, aplicações de atividades, feedbacks e geração de sugestões — é registrada em **LOG_ACAO** com identificação do usuário, tipo de operação, entidade afetada e timestamp, garantindo rastreabilidade completa conforme RNF-006 e RNF-007.

9. Volume e Desempenho

Esta seção apresenta as principais características de dimensionamento do Educa+, com estimativas baseadas nos requisitos não funcionais do projeto e no contexto de implantação piloto em até três escolas de ensino médio. Os valores foram derivados a partir do RNF-002 (desempenho nas consultas), RNF-006 (rastreabilidade) e do perfil esperado de uso do sistema em horário letivo.

Item	Estimativa	Base / Justificativa
Usuários cadastrados	1.500 usuários (1.350 alunos + 150 professores)	3 escolas com aproximadamente 500 alunos e 50 professores cada.
Usuários simultâneos — normal	150 usuários	Cerca de 10% dos usuários ativos durante horário de aula regular.
Usuários simultâneos — pico	300 usuários	Múltiplas turmas respondendo atividades avaliativas ao mesmo tempo.
Requisições por segundo — normal	30 req/s	Navegação, consultas de desempenho, feedbacks e sugestões de conteúdo.

<Nome do grupo>

Requisições por segundo — pico	80 req/s	Aplicação simultânea de atividades por várias turmas.
Tempo de resposta máximo	3 segundos em 90% das requisições	Conforme RNF-002 — desempenho nas consultas pedagógicas.
Volume inicial de dados	~300 MB	Cadastros de usuários, turmas, disciplinas, conteúdos BNCC, questões, atividades e primeiras respostas.
Crescimento estimado	~80 MB por semestre	Acúmulo de respostas de atividades, logs de ações, registros de desempenho e sugestões geradas.
Retenção de logs	12 meses	Conforme RNF-006 — rastreabilidade de todas as ações relevantes do sistema.
Backup	Diário	Garantia de recuperação de dados acadêmicos em caso de falha.
Disponibilidade esperada	95% em horário letivo	Segunda a sexta-feira, das 07h às 22h, período de uso ativo do sistema.

10. Referências

KRUCHTEN, Philippe. **The 4+1 view model of architecture**. IEEE Software, v. 11, n. 6, p. 42-50, nov. 1994.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR ISO/IEC 25010**: sistemas e software — requisitos e avaliação de qualidade de sistemas e software (SQuaRE) — modelos de qualidade de sistema e software. Rio de Janeiro: ABNT, 2011.

BRASIL. Ministério da Educação. **Base Nacional Comum Curricular**. Brasília: MEC, 2018. Disponível em: <http://basenacionalcomum.mec.gov.br>. Acesso em: mar. 2026.

NESTJS. **NestJS documentation**. Versão 10. 2024. Disponível em: <https://docs.nestjs.com>. Acesso em: mai. 2026.

REACT NATIVE. **React Native documentation**. Versão 0.73. 2024. Disponível em: <https://reactnative.dev/docs/getting-started>. Acesso em: mai. 2026.

EXPO. **Expo documentation**. SDK 50. 2024. Disponível em: <https://docs.expo.dev>. Acesso em: mai. 2026.

PRISMA. **Prisma documentation**. Versão 5. 2024. Disponível em: <https://www.prisma.io/docs>. Acesso em: mai. 2026.

MYSQL. **MySQL 8.0 reference manual**. Oracle Corporation, 2024. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en>. Acesso em: mai. 2026.

FIREBASE. **Firebase documentation**. Google LLC, 2024. Disponível em: <https://firebase.google.com/docs>. Acesso em: mai. 2026.

WINSTON. **Winston logger documentation**. Versão 3. 2024. Disponível em: <https://github.com/winstonjs/winston>. Acesso em: mai. 2026.